

CS201 Midterm 2

7 December 2023

Olcay Taner YILDIZ

Abstract—Write only ONE function per question. Do not check the input, assume that the input is correct. Unless noted, (i) the input data structure is not empty, (ii) time complexity of the algorithm does not matter, (iii) you can not use extra data structures.

I. QUESTION (BINARY SEARCH TREE)

Write the recursive method

```
int averages()
```

in `TreeNode` class which returns the number of nodes in the tree that satisfy the following property: The node's key is the average of its children (left and right children).

II. QUESTION (BINARY SEARCH TREE)

Write a non-recursive method

```
int sumOfPath(String path)
```

in `Tree` class, which sums the keys on the path, where the path is defined by the parameter `path` as follows: (i) Path consists of 0's and 1's such as 10011. (ii) A 1 represents to go right, a 0 represents to go left. If the path is 1011, you start from root, you go first right, then left, then right, then right. If the path is 001, you start from root, you go first left, then left, then right. You will use `charAt` function in strings.

III. QUESTION (QUEUE)

Write the method

```
void removeOddIndexed()
```

which removes only the odd indexed (1, 3, ...) elements from the queue. The first element has index 1. You are only allowed to use `enqueue`, `dequeue`, `isEmpty` functions. **You should use external queue.** You are not allowed to use any queue attributes such as `first`, `last`, `array` etc. You can solve the question with any queue implementation.

IV. QUESTION (QUEUE)

Write the methods

```
Element dequeue(int k)
```

```
Node dequeue(int k)
```

which dequeues data as the k 'th element from the first. `dequeue(1)` is equal to the original `dequeue`, that is, the first element has index 1. You are not allowed to use any queue methods and any external structures (arrays, queues, trees, etc). You are allowed to use attributes, constructors, getters and setters. Write the method for both array and linked list implementations.

V. QUESTION (HASHING)

a. Write the method

```
int numberOfClusters()
```

that finds the number of clusters in hash table. A cluster is a contiguous group of non-null elements in the array.

b. Write the method

```
boolean perfectMap()
```

that returns true if the hash table contains one node at maximum per linked list in separate chaining, otherwise it returns false.

VI. QUESTION (HASHING)

Write a static method in `Hash` class

```
int numberOfExtras(int[] array)
```

that takes an array of integers as a parameter and counts the number of extra elements in the array. **Your method should run in $\mathcal{O}(N)$ time, where N is the size of the array.** Use hashing.

1 4 2 5 2 4 3 4 $\rightarrow$ 3 extras (two 4, one 2)

2 1 2 1 2 3 1 2 1 2 $\rightarrow$ 7 extras (four 2, three 1)

1 1 1 1 1 1 $\rightarrow$ 5 extras (five 1)